GHA on-boarding user guide

This document illustrates step by step process to on-board any repo/micro-service to GHA CD.

<Example snippets correspond to cvs-health-source-code repos, but its the same for cvs-health-pcw-source-code repos too, its just that the org name has to be changed.>

⚠ **Please review all the steps and then start with the implementation**

⚠ **For teams migrating from Harness, do not use the existing manifest repo.**

The following are the steps to be completed in order to on-board any repo/micro-service to GHA CD

1. Creating **.github/workflows/cd.yaml** file at the root.
2. Creating **deploy-configs** folder to host **env./namespace** specific helm **values/configs** for the deployment.
3. Creating manifest repo if does not exists along with cutting out a branch on the namespace name.
4. Creating **namespace-configs/<namespace>.yaml** at the respective branch in the manifest repo.
5. Setting up Argo credentials as Repository secrets and Variables.
6. Setting up environments with cluster name, type and manifest repo details.
7. Trigger the deployment.

REVIEW THE FOLLOWING STRUCTURES:

YOUR APPLICATION REPOSITORY:

```
your-application-repo/
├── .github/
│   └── workflows/
│       └── cd.yaml                    # CD workflow (copy from
template)                              # Your application source code
│
├── deploy-configs/
│   ├── [common]/               # e.g., dev, staging, qa
│   │   └── common.yaml
│   │
│   ├── [namespace1]/               # e.g., dev, staging, qa
│   │   └── values/
│   │       ├── v1.yaml               # Helm values version 1
│   │       └── v2.yaml               # Helm values version 2
(optional)
│   │   └── config/
│   │       ├── config1.yaml               # configs to be mounted
│   │       └── config2.json
│   │
│   ├── [namespace2]/               # e.g., prod
│   │   └── values/
│   │       ├── v1.yaml               # Helm values version 1
│   │   └── config/
│   │       ├── config1.yaml               # configs to be mounted
│   │       └── config2.json
```

Note: Anything commit/push to v1.yaml or v2.yaml, will trigger the deployment.

MANIFEST REPOSITORY STRUCTURE:

```
[cluster-name]-[cluster-type]_manifests/  # Custom name is also allowed
<namespace-branch>
├── namespace-configs/
|   ├── [namespace].yaml          # Namespace configuration
```

STEP 1:

Expand step 1:

Create .github/workflows/cd.yaml file at the root of your repo with the following snippet.

```
name: App Deployment

on:
  workflow_dispatch:
  push:
    branches: [main]
    paths: ['**/values/v1.yaml']

permissions:
  id-token: write
  contents: write
  pull-requests: write

jobs:
  detect-changes:
    runs-on: cvs-linux-self-hosted //self-hosted, for pcw org.
    outputs:
      CHANGED_ENVIRONMENTS: ${{ steps.set-
matrix.outputs.CHANGED_ENVIRONMENTS }}
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Detect changes
        id: changes
        uses: dorny/paths-
filter@de90cc6fb38fc0963ad72b210f1f284cd68cea36
        with:
          base: ${{ github.ref_name }}
          filters: |
            test-dev_v1: '**/deploy-configs/test-
dev/values/v1.yaml'
            test-prod_v1: '**/deploy-configs/test-
prod/values/v1.yaml'

      - name: Build Environment Matrix
        id: set-matrix
        run: |
          ENVS=()

          if [ "${{ steps.changes.outputs.test-dev_v1 }}" = "true"
]; then
            ENVS+=("{\"ENV_NAME\":\"test-
dev\",\"HELM_FILE_VERSION\":\"v1\"}")
          fi

          if [ "${{ steps.changes.outputs.test-prod_v1 }}" = "true"
]; then
```

```
            ENVS+=("{\"ENV_NAME\":\"test-prod\",\"HELM_FILE_VERSION\":\"v1\"}")
          fi

          if [ ${#ENVS[@]} -gt 0 ]; then
            JSON=$(IFS=,; echo "[${ENVS[*]}]")
            echo "CHANGED_ENVIRONMENTS=$JSON" >> $GITHUB_OUTPUT
          else
            echo "CHANGED_ENVIRONMENTS=[]" >> $GITHUB_OUTPUT
          fi

  deploy:
    uses: cvs-health-source-code/gitops-cd-workflow/.github/workflows/helm-deploy.yaml@v1
    needs: detect-changes
    if: needs.detect-changes.outputs.CHANGED_ENVIRONMENTS != '[]'
    strategy:
      fail-fast: false
      matrix:
        environment: ${{ fromJson(needs.detect-changes.outputs.CHANGED_ENVIRONMENTS) }}
    secrets: inherit
    with:
      app-name: "test-app"
      environment: ${{ matrix.environment.ENV_NAME }}
      helm-template-repo: ${{ vars.HELM_TEMPLATE_REPO }}
      helm-file-version: ${{ matrix.environment.HELM_FILE_VERSION }}
      helm-template-branch: ${{ vars.HELM_TEMPLATE_BRANCH }}
      canary-weight: "0"
      lob: "devex"
      change-description: "test change"
      rfc-short-desc: "test-description"
      change-domain: "Health Care Delivery(HCD)"
      change-impacted-website: "CVS.COM"
```

** For PCW org. use the following in cd.yml

```
jobs:
  detect-changes:
    runs-on: self-hosted
```

And:

```
deploy:
    uses: cvs-health-pcw-source-code/gitops-cd-workflow/.github/workflows/helm-deploy.yaml@main
```

You can now edit few parameters in it, which works according to your micro-service/deployment.

1. The app-name is given as test-app, which should be replaced with your micro-service name.
2. test-dev and test-prod are used as an example namespaces, which should be corrected accordingly and add additional namespaces if required.
3. canary-weight will remain “0”, unless if any deployment requires traffic split between live version and deploy version.
4. The lob value changes from team to team, this parameter decides the respective approval group for the production deployment.

Expand this for the list of lobs and respective prod approval group name

account: "account-gha-cd-approver"
retail: "retail-gha-cd-approver"
pharmacy: "pharmacy-gha-cd-approver"
sre: "pharmacy-gha-cd-approver"
health: "health-gha-cd-approver"
messaging: "martech-gha-cd-approver"
iot: "devex-gha-cd-approver"
ciam: "devex-gha-cd-approver"
activehealth: "martech-gha-cd-approver"
devex: "devex-gha-cd-approver"
ivr: "ivr-gha-cd-approver"

Note: These approval groups are GitHub teams names (for pcw, please search under pcw org.), the respective person should be present in that teams group to approve there pipeline during production deployment, if someone is not present, PROD-SUPPORT TEAM can add members to it.

5. change-description and rfc-short-desc can be ignored for non prod deployments but, when the micro-service is going for the prod release, the respective team has to update these parameters first with proper release notes, and then trigger the deployment next (any push/commit v1.yaml will trigger the deployment).
6. Below are the list of change-domains and impacted sites, if you have multiple, please chose one among them.

change-domain	***change-impacted-website***
Enterprise	❤CVS - Online Drugstore, Pharmacy, Prescriptions & Health Information
Health Care Business(HCB)	❤CVS Caremark Home
Lower Level Environment(LLE)	http://AETNA.COM
Pharmacy Services(PS)	SPECIALTY
Pharmacy & Consumer Wellness(PCW)	❤View Health Records and Find Care | CVS
Health Care Delivery(HCD)	myactivehealth.com

STEP 2:

Expand step 2:

Create deploy-configs folder to host env./namespace specific helm values/configs for the deployment.

1. Under the deploy-configs folder, we first create common/common.yaml, this holds the common configs for the deployment irrespective of the environments/namespaces.

Below is the sample snippet:

```
#app related data
app:
  name: test-app //update with your app name
  port: 12345 //update with your port number
  gitRepoName: cvs-health-source-code/test-repo //update with your repo path
  enableOpenTelemetry: "true" //optional can be commented if not required.


#livenessProbe details
livenessProbeDetails:
  probePath: "/microservices/test-app/healthz" //update with your probe path
  initialDelaySeconds: 30

#readinessProbe details
readinessProbeDetails:
  probePath: "/microservices/test-app/healthz" //update with your probe path
  initialDelaySeconds: 30

#startupProbe details
startupProbeDetails:
  probePath: "/microservices/test-app/healthz" //update with your probe path
  initialDelaySeconds: 60
  periodSeconds: 10
  failureThreshold: 30

#custom pod annontations //optional this section can be commented if not required.
customPodAnnotations: |
  name: "{{ .Values.app.name }}"
  prometheus.io/port: "{{ .Values.app.port }}"
  prometheus.io/path: "/actuator/metrics"
  instrumentation.opentelemetry.io/inject-java: "opentelemetry-operator/my-instrumentation"
  instrumentation.opentelemetry.io/container-names: "{{ .Values.app.name }}"

istioHttp: |
  {{- if .Values.istio.cookieBasedRouting }}
  # Cookie-based routing: Route canary=on requests to new deployment
  - match:
    - headers:
        cookie:
          regex: ^(.*?;)?\s*(canary=on)(;.*)?$
      uri:
        prefix: /depservices/
    route:
    - destination:
        host: {{ .Values.app.name }}.{{ .Values.appData.nameSpace }}.svc.cluster.local
        port:
          number: {{ .Values.app.port }}
        subset: {{ regexReplaceAll "\\W+" .Values.istio.canaryAppVersion "-" }}
      weight: 100
  {{- end }}
  # Default routing for all scenarios (cookie-based default, weight-based, rolling)
```

```
  - match:
    - uri:
        prefix: /depservices/
    route:
    {{- if and (ne .Values.istio.currentAppVersion
.Values.istio.canaryAppVersion) (not
.Values.istio.cookieBasedRouting) }}
    # Weight-based canary: Split traffic between current and canary
    - destination:
        host: {{ .Values.app.name }}.{{ .Values.appData.nameSpace
}}.svc.cluster.local
        port:
          number: {{ .Values.app.port }}
        subset: {{ regexReplaceAll "\\W+"
.Values.istio.currentAppVersion "-" }}
      weight: {{ .Values.istio.prodWeightage }}
    - destination:
        host: {{ .Values.app.name }}.{{ .Values.appData.nameSpace
}}.svc.cluster.local
        port:
          number: {{ .Values.app.port }}
        subset: {{ regexReplaceAll "\\W+"
.Values.istio.canaryAppVersion "-" }}
      weight: {{ .Values.istio.canaryWeightage }}
    {{- else }}
    # Single version routing (cookie-based default OR rolling
deployment)
    - destination:
        host: {{ .Values.app.name }}.{{ .Values.appData.nameSpace
}}.svc.cluster.local
        port:
          number: {{ .Values.app.port }}
        subset: {{ regexReplaceAll "\\W+"
.Values.istio.currentAppVersion "-" }}
      weight: 100
    {{- end }}
```

2. Now its time to create namespace specific folder and respective values and config folders under it, if your app do not require any additional configs to be passed then ignore creating config folder.

   Sample v1.yaml:

```
#namespace, app version and image details...
appData:
  nameSpace: test-dev //update correct one
  dockerHub: cvsh.jfrog.io
  dockerImageGroupName: cvsdigital-docker/ghemu/... //update
correct one
  dockerImageName: test-app //update correct one
  dockerImageTag: 1.0.0 //update correct one

#hpa settings - min and max replicas and minAvailable for PDB
hpaSettings:
  minAvailable: 1
  minReplicas: 1
  maxReplicas: 1
  averageUtilization: 80

#Updates domain portal //remove this section, if not required
# deployValidations:
#   - name: i90-domain
#     repo_name: i90-domain-harness-automation-deploy

# Pod resource requests and limits
resources:
  requests:
```

```
    cpu: "100m"
    memory: "250Mi"
  limits:
    cpu: "200m"
    memory: "350Mi"

# environment variables for app - secret values come from vault
envVars: |
  - name: TEST_KEY
    value: TEST_VALUE
  - name: TEST_KEY
    value: TEST_VALUE

#configMap BasePath //remove this section, if not required
configBasePath: /opt/digital/... //update correct one

# ConfigMaps to be Created //remove this section, if not required
configMap:
  test-app-application.yaml: |
    {{- .Files.Get "config/test-app-application.yaml" }}
  test-app-error.json: |
    {{- .Files.Get "config/test-app-error.json" }}

# env secrets to be referred as ENV variabes //remove this section, if not required
envSecrets: |
  - secretRef:
      name: secret-folder-name1
      optional: false
  - secretRef:
      name: secret-folder-name2
      optional: false

extraVolumeMounts: | //remove this section, if not required
  - name: cert-folder-name
    mountPath: /opt/digital/microservices/test-app/certs
    readOnly: true

extraVolumes: | //remove this section, if not required
  - name: cert-folder-name
    secret:
      secretName: cert-folder-name
      optional: false

# Cloud sql sidecar enabled //remove this section, if not required or fill in correct values
cloudSqlSidecar:
  enabled: true
  instanceName: digital-dev-iot:us-east4:test-postgresql-dev=tcp:4321
  requestCpu: "500m"
  requestMemory: "1Gi"
  limitCpu: "500m"
  limitMemory: "1Gi"

# Cloud sql sidecar enabled //remove this section, if not required or fill in correct values
workloadIdentity:
  gcpProject: digital-dev-iot-test
  serviceAccountName: cloudsql-digital-iot-test
  serviceAccountEmail: cloudsql-digital-iot@digital-dev-test-iot.iam.gserviceaccount.com
```

3. If you have mentioned configs in your v1.yaml then create config folder accordingly and have the respective files under it.

STEP 3:

Expand step 3:

1. Create manifest repo if does not exists (add a readme file - optional/recommended).

   We recommend to create manifest repo with the following naming standard:
   [cluster-name]-[cluster-type]_manifests
   eg: pharma-dev-gke_manifests, prod-pharmacy-rke_manifests

   Note:
   1. Manifest repo will be unique to the cluster, we do not recommend multiple manifest repos to one cluster. All the namespaces under that cluster, will be as branches under the manifest repo.
   2. Custom namespaces are also allowed for naming manifest repo, but it has to be passed as an additional parameter for the deployment pipeline, which will be discussed down the line.

2. Cut a branch from **main** in the manifest repo with the namespace name and create **namespace-configs/[namespace].yaml** file in it.

   sample content:

```
#argocd related data
argo:
  project: default
  manifestRepo: "https://github.com/cvs-health-source-code/test-manifest.git" //update repo url .git
  notifyEmailList: "abc@cvshealth.com, user2@cvshealth.com"

#istio related values
istioInternalHosts: | // update your cluster internal dns values
  - "sit-dgtl-pbm.internal.cvshealth.com"
  - "other-internal-dns"

istioExternallHosts: | // update your cluster external dns values
  - "external-dns.cvshealth.com"

#istio Host details for Istio auth // update the list
istioHosts: '["sit-dgtl-pbm.internal.cvshealth.com", "other-internal-dns", "external-dns.cvshealth.com"]'

#namespace resources default and limits
namespaceDetails:
  defaultMemory: "350Mi"
  defaultCpu: "500m"
  maxMemory: "2Gi"
  maxCpu: "2"

jwtVaultClusterScope: // optional, remove this section if not required
  vaultURL: "https://vault-nonprod.cvshealth.com"
  tenant: "gcp/rke-project-name"
  kubernetesCluster: "gcp/rke-cluster-name"
  externalSecrets:
    i90-secrets:
      path: kv-v2/app/i90-apollo/data/test

#vault Details at namespace level
jwtVault: // optional, remove this section if not required
  vaultURL: "https://vault-nonprod.cvshealth.com"
  tenant: "gcp/rke-project-name"
  kubernetesCluster: "gcp/rke-cluster-name"
```

```
  externalSecrets:
    - secret-folder-name1
    - secret-folder-name2
    - secret-folder-name3
  externalSecretsBase64:
    - cert-folder-name1
```

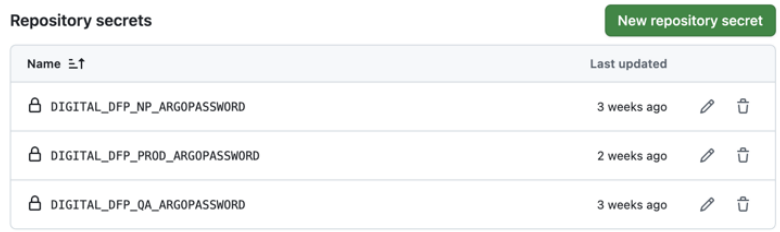
STEP 4:

Expand step 4:

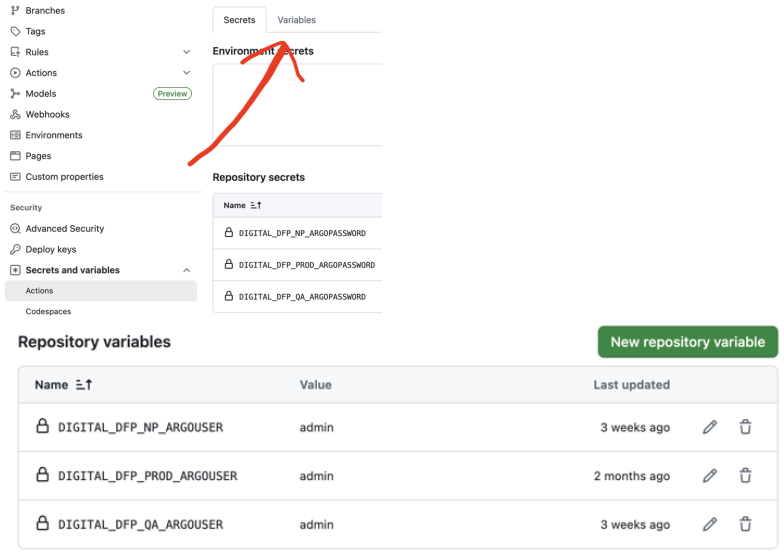
Now go to setting of your source-code repo (one used in step 1 & 2), if you do not see setting over your repo that mean you do not have admin access over your repo.

1. Navigate to settings/ Secrets and variables (left-side)/ Actions: Under Repository secrets, please add your cluster argo admin password by clicking on new repository secret.

   Eg: Lets say I am trying to deploy to a dev namespace, which is/should-be part of **pharma-dev** cluster, then I create a secret with key as **PHARMA_DEV_ARGOPASSWORD** and value is your argo admin password.
   Note: For Prod, its prod support team who will add it, for the moment, and maintain same format of naming as in example screen shots.



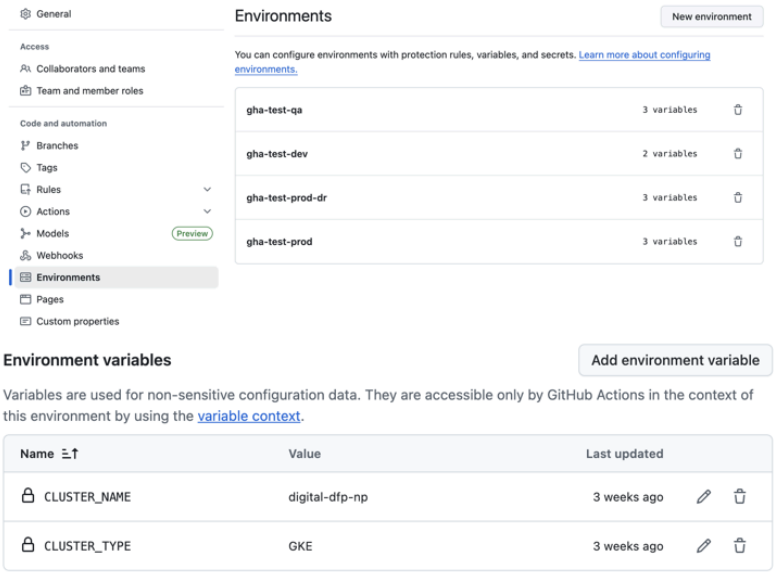
2. Now hover to variables section and add your cluster argo admin user name under repository variables section.



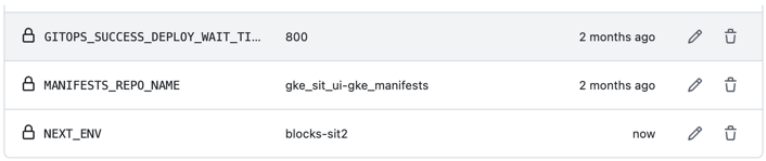
3. Now Click on Environments to your left and create your namespaces under it (delete if there is any unwanted) with following parameters under it.

Note: When you create your environment/namespace, fill in the environment variables section with CLUSTER_NAME & CLUSTER_TYPE which are mandatory.

Please follow as in example screen shots:



Additionally you can add few other variables such as:
MANIFESTS_REPO_NAME - incase you have custom manifest repo name.
GITOPS_SUCCESS_DEPLOY_WAIT_TIME - incase you want pipeline to wait for more time, in the argo sync and app health validation section. (default is 600 seconds which should be good, you can custom entry values in seconds if you want it more - not recommended)
NEXT_ENV - incase you want the automatic deployment trigger in other namespace.
(eg: Lets say I updated my docker image tag in dev namespace v1.yaml file then deployment will get automatically trigger for dev and if its successful, since I added NEXT_ENV value as qa, then the pipeline will update the same docker image tag in qa namespace v1.yaml which triggers qa deployment. It only updates docker image tag, not the rest of configs)



STEP 5:

Expand step 5:

Its all done, now any trigger to your v1.yaml or v2.yaml (optional) will trigger the deployment.

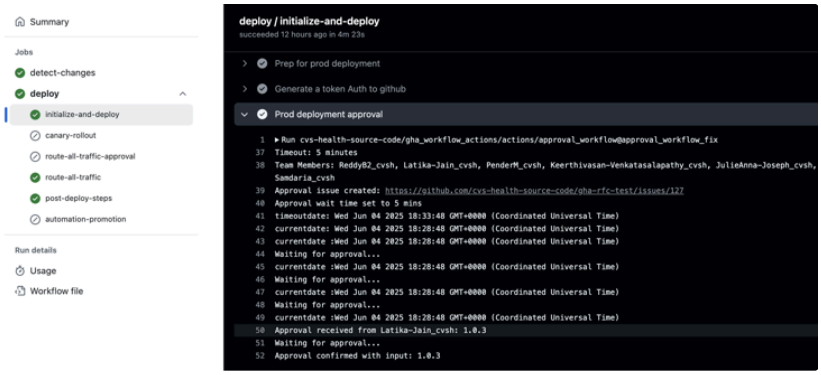
For Vault configurations:

Using Vault with Harness via native JWT

Production pipeline approval process:

Expand Production pipeline approval process:

Unlike the non-prod deployments the pipeline will wait for 5 minutes (it can not be increased) at Prod deployment approval stage, for pipeline to receive approval, else pipeline aborts.



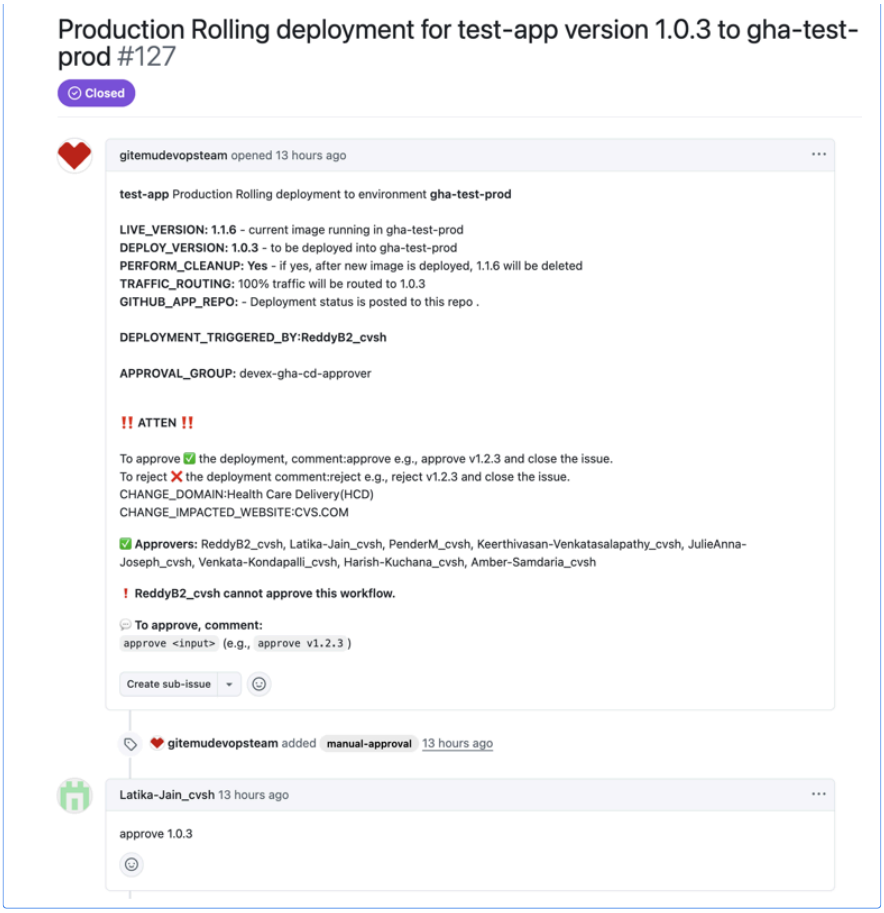
We advise you to have the pipeline being monitored, as at this stage there will be an GithubIssue for approval created and published as a https:// link.

That link has to be passed to the respective member who has access to approve this prod deployment.

To approve this deployment, the respective member as to comment **approve** keyword along with deploy version.

Example:

Since the deploy version in here is 1.0.3, the member authorized to approve has commented **approve 1.0.3**



Caution:

1. Please add path-ignore section in your ci.yml workflow if you have one, in your repo. So that for every CD specific update the CI pipeline will not trigger.
2. This will not work for the mono repo style, meaning one repo hosting multiple micro-service deployments across multiple environments, the process is almost the same with little change (will publish a doc on it).
3. For teams migrating from **Harness, do not use the existing manifest repo or namespaces**.
4. Make sure you have your cluster listed in here: https://github.com/cvs-health-source-code/stargate_allow_list/blob/main/cd/gha_cd/cd_env_config.yaml (source code org.) https://github.com/cvs-health-pcw-source-code/pcw_allow_list/blob/main/cd/gha_cd/cd_env_config.yaml (pcw org.)

Create new manifest repo and deploy your micro-services to new namespace and once the deployment is validated in new namespace, then the respective references in the old-namespace can be delete. Just an fyi, new namespace comes up with new vault engine

#pcw-devex-devsecops-engagement-team